

Esc. Tecnica Rocha — ANEP, Consejo de Educacion Tecnico Profesional (UTU)
3.º BT Tecnologias de la Informacion
Programacion Full-Stack

UTalent

Buscador de oportunidades laborales en Tecnologias de la Informacion para estudiantes y egresados de UTU

Cuarto Practico — 2026
Estudiante: Lucas Techera (C.I. 4.999.390-1)
Rocha, setiembre de 2026

Indice

- 1. Propuesta y arquitectura..... 3
 - 1.1 Proyecto elegido..... 3
 - 1.2 Eleccion del stack: Laravel..... 3
 - 1.3 Patron de arquitectura: MVC + Strategy + Repository..... 3
 - 1.4 Diagrama de clases..... 4

1. Propuesta y arquitectura

1.1 Proyecto elegido

Buscador de oportunidades laborales en Tecnologías de la Información dirigido a estudiantes y egresados técnicos de UTU. El sistema permite buscar ofertas por título de puesto (por ejemplo, “Programador” o “Soporte Técnico”) y filtrar por sector (público/privado), visibilidad del salario, modalidad de postulación y departamento, agregando resultados de distintas fuentes: el portal estatal Uruguay Concurso (empleo público) y portales privados de empleo.

1.2 Elección del stack: Laravel

Se descarto PHP/SQL vanilla en favor de Laravel por los siguientes motivos:

- El foco del práctico es la extracción y consulta de datos, no la infraestructura. El proyecto necesita colas de trabajo (para actualizar ofertas en segundo plano sin bloquear al usuario), un scheduler (para relanzar el scraping periódicamente), un cliente HTTP para invocar un modelo de lenguaje, y un ORM para modelar relaciones entre fuentes, ofertas y sinónimos de búsqueda. Laravel provee todo esto de fábrica (Queues, Scheduler, Http::, Eloquent), dejando el tiempo disponible para la lógica de scraping y normalización, que es el objetivo pedagógico real.
- Consistencia de convenciones: al tener varias fuentes de datos heterogéneas, conviene una estructura de carpetas y convenciones ya establecidas en vez de definirlas desde cero.
- Valor de aprendizaje adicional: el grupo ya cuenta con experiencia en PHP/SQL vanilla en otro proyecto, por lo que este práctico suma un framework ampliamente usado en la industria, en línea con el aporte al proyecto de egreso que pide el ejercicio 3.

1.3 Patrón de arquitectura: MVC + Strategy + Repository

El proyecto usa el MVC que Laravel impone por convención (rutas → controladores → modelos Eloquent → vistas Blade) como esqueleto general, pero el MVC puro no resuelve bien un problema central de este proyecto: obtener datos de fuentes externas heterogéneas y cambiantes sin acoplar esa lógica al resto de la aplicación. Por eso se suman dos patrones adicionales:

- Strategy: cada fuente de empleo (Uruguay Concurso, portal privado A, portal privado B) implementa una interfaz común FuenteEmpleoInterface. Agregar, quitar o reparar una fuente no requiere tocar el resto del sistema (principio abierto/cerrado): si un sitio cambia su HTML, solo se ajusta su estrategia concreta.
- Repository: el acceso a la persistencia (ofertas, fuentes, sinónimos) se aísla detrás de interfaces (OfertaRepositoryInterface, etc.), separando de donde vienen los datos scrapeados de cómo se guardan y consultan localmente. Esto también permite que la búsqueda no dependa de scrapear en vivo en cada request: los resultados se persisten y refrescan por jobs en segundo plano, haciendo el sistema tolerante a que una fuente falle o cambie de estructura.
- Un Service (BuscadorService) orquesta las estrategias, delega la normalización de texto libre a un servicio de IA (NormalizadorIA) y persiste vía los repositorios.

1.4 Diagrama de clases

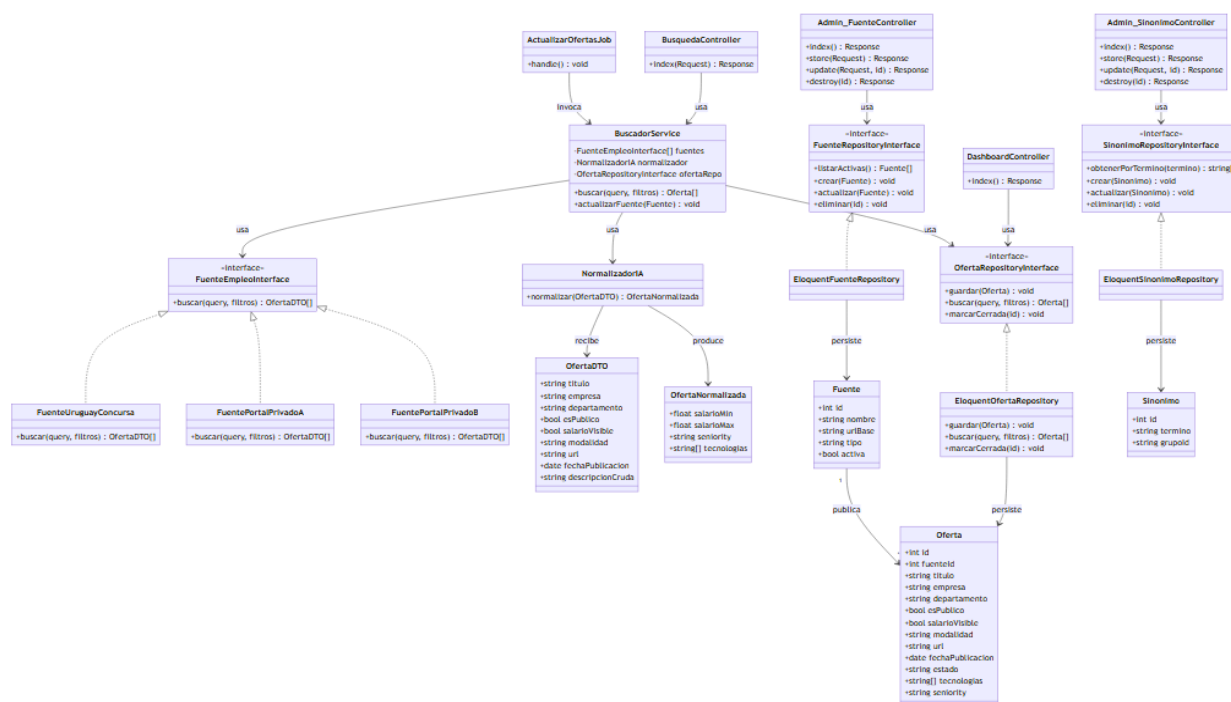


Diagrama de clases: interfaces de fuentes (Strategy), servicio orquestador, normalizador de IA, repositorios (Repository) y modelos. FuenteEmpleadoInterface y sus implementaciones son el punto de extensión: sumar un portal nuevo es crear una clase mas, sin tocar BuscadorService. NormalizadorIA es el unico punto de contacto con el modelo de lenguaje (documentado en la seccion de uso de IA): recibe el texto crudo de una oferta y devuelve campos estructurados (salario, seniority, tecnologias). Los tres repositorios (Oferta, Fuente, Sinonimo) son el ABM real del sistema, reemplazando el CRUD de usuarios que la consigna explicitamente no pide para esta opcion: se administra el propio motor de busqueda (fuentes activas y sinonimos de busqueda) en lugar de cuentas de usuario. ActualizarOfertasJob corre en segundo plano (cola y scheduler de Laravel), evitando scrapear en vivo en cada busqueda del usuario y aislando fallas de una fuente puntual.